Exhaustive Deep Dive: Validate Last Week Timesheet Entry Flow

This document provides a highly detailed, step-by-step breakdown of the "Validate Last Week Timesheet Entry" functionality. It traces the entire execution path beginning from the moment a user interacts with the UI in the View, through the client-side JavaScript AJAX request, into the backend ASP.NET Core Controller, passing through the populated ViewModel, diving deep into the Service layer (covering both historical commented code and active logic), and finally detailing the Database entity interactions.

---

1. The View Layer (User Interface & HTML Structure)

The entry point for this flow begins in the user's browser, specifically on the Timesheet capture page. The HTML table that holds the timesheet entries is generated by the `_TimesheetTableLoad.cshtml` Razor partial view.

The Razor View Setup

The view iterates over the `Model.TimesheetEntryDetails` collection to render editable rows. Each row contains data attributes to store values that define the entry.

```
@for (var i = 0; i <= Model.TimesheetEntryDetails.Count - 1; i++)
{
    var clsName = "timeEntryTable-" + i;
    <tr class="timeEntryTable @clsName">
        <td></td>
        <!-- Hidden fields storing crucial identifiers -->
        <td hidden class="TEGroupTime" data-selVal="@Model.TimesheetEntryDetails[i].Group
Time"></td>
        @if (@Model.TimesheetEntryDetails[i].Id != 0)
        {
            <td hidden class="TEId" data-selVal="@Model.TimesheetEntryDetails[i].Id"></t
d>
            <td hidden class="TECapturedDateTime" data-selVal="@Model.TimesheetEntryDetai
ls[i].CapturedDateTime"></td>
            <td hidden class="TECapturedBy" data-selVal="@Model.TimesheetEntryDetails[i].
CapturedBy"></td>
            <td hidden class="TEUserId" data-selVal="@Model.TimesheetEntryDetails[i].User
Id"></td>
        }
        <td hidden class="TEApprovalStatus" data-selVal="@Model.TimesheetEntryDetails[i].
ApprovalStatus"></td>

        <!-- Visible input columns -->
        <td class="TECustomerName">@Html.DisplayFor(m => Model.TimesheetEntryDetails[i].C
ustomerName)</td>
        <td data-ctype="pckr" class="TEProjectName" data-selVal="@Model.TimesheetEntryDet
ails[i].ProjectId">...</td>
        <td data-ctype="pckr" class="TETaskName" data-selVal="@Model.TimesheetEntryDetail
s[i].ProjectWbId">...</td>
        <td data-ctype="pckr" class="TEBillability" data-selVal="@Model.TimesheetEntryDet
ails[i].Billability">...</td>
        <td data-ctype="timepckr" class="TEStartTime">@Html.DisplayFor(m => Model.Timeshe
```

```
etEntryDetails[i].StartDatetimeString)</td>
        <td data-ctype="timepckr" class="TEEndTime">@Html.DisplayFor(m => Model.Timesheet
EntryDetails[i].EndDatetimeString)</td>
        <td class="TEHours">@Html.DisplayFor(model => Model.TimesheetEntryDetails[i].Hour
s)</td>
        <td data-ctype="mtxtbox" class="TEComments" data-selVal="@Model.TimesheetEntryDet
ails[i].Comment">...</td>
        <td hidden class="TEOverlapTimesheet" data-selVal="@Model.TimesheetEntryDetails
[i].OverlapTimesheet"></td>
    </tr>
}
```

Key Triggers

When the view is loaded, a JavaScript library named `BSTable` is initialized on this table. It appends Action buttons (Edit, Delete, Save/Accept, Cancel) to each row. When a user modifies a row (changing hours or project) and clicks the "Accept/Save" button (`<button id="bAcep">`), the `BSTable` framework triggers the `onEdit` callback, which is explicitly bound to the `updateTimeEntry(button)` JavaScript function.

---

2. The Client-Side Logic (JavaScript & AJAX)

Once the user clicks "Save" on a row, the `updateTimeEntry` function takes over in `_TimesheetTableLoad.cshtml`.

Extracting Data

The script first identifies the specific row using the `button` element and extracts values from the `<td>` sections utilizing jQuery data attributes (`data('selval')`) inner texts.

```
function updateTimeEntry(button) {
    var tdElement = $(button);

    // Frontend validations (Checking for empty Comments, missing Tasks, or low Hours)
    if (tdElement.find('.TEProjectName').html() == '') { /* show error & return */ }
    if (parseFloat(tdElement.find('.TEHours').html()) < 0.25) { /* show error & return */
 }
    if (tdElement.find('.TETaskName').html() == '') { /* show error & return */ }

    // Date parsing
    var GroupTime = moment($('tbody tr td.TEGroupTime').data('selval').split(' ')[0], "DD
/MM/YYYY").isValid() ?
                    moment($('tbody tr td.TEGroupTime').data('selval').split(' ')[0], "DD
/MM/YYYY").toDate() :
                    new Date($('tbody tr td.TEGroupTime').data('selval').split(' ')[0]);
```

Constructing the Payload

The JavaScript constructs either a `tsModel` (for new entries) or a `tmModel` (for updating existing entries) object containing all the extracted row data. This structure perfectly aligns with the C# `TimeSheetEntryViewModel`.

```
    var tsModel = {
        ProjectId: tdElement.find('.TEProjectName').data('selval'),
        ProjectWbId: tdElement.find('.TETaskName').data('selval'),
```

```
        CustomerSiteId: tdElement.find('.TESiteName').data('selval'),
        StartDatetime: $('tbody tr td.TEGroupTime').data('selval').split(' ')[0] + " " +
tdElement.find('.TEStartTime').html(),
        EndDatetime: $('tbody tr td.TEGroupTime').data('selval').split(' ')[0] + " " + td
Element.find('.TEEndTime').html(),
        IsBillable: tdElement.find('.TEBillability').html() == "Billable" ? true : false,
        Comment: tdElement.find('.TEComments').text(),
        Hours: tdElement.find('.TEHours').html(),
        CapturedDateTime: tdElement.find('.TECapturedDateTime').data("selval"),
        CapturedBy: tdElement.find('.TECapturedBy').data("selval"),
        UserId: tdElement.find('.TEUserId').data("selval"),
        OverlapTimesheet: tdElement.find('.TEOverlapTimesheet').data("selval"),
        __RequestVerificationToken: $('#frmTimeEntry input[name="__RequestVerificationTok
en"]').val(),
    };

    var tmModel = { /* Same as above, but adds the Id property */ Id: tdElement.find('.TE
Id').data("selval") };

    var ModelSend = tdElement.find('.TEId').data("selval") != 0 && tdElement.find('.TEId
').data("selval") != undefined ? tmModel : tsModel;
```

Dispatching the AJAX Request

With the JSON object ready, the browser issues a POST request to the Controller.

```
    $.ajax({
        type: "POST",
        url: "/Timesheet/TimesheetEntryDetails",
        data: ModelSend,
        success: function (resultData) {
            getTimesheetDetailsDetails(); // Refresh the grid
            if (resultData != "false" && resultData.status) {
                showToastr("success", resultData.message);

                // IMPORTANT: This block explicitly looks for the "warning" keyword retur
ned
                // by the ValidateLastWeekTimesheetEntry logic
                if (resultData.message && resultData.message.toLowerCase().includes('warn
ing')) {
                    showToastr("warning", resultData.message);
                }
            } else {
                showToastr((resultData.toastrType == "" ? "error" : resultData.toastrTyp
e), resultData.message);
            }
        },
        error: function (resultData) {
            showToastr("error", resultData);
        }
    });
}
```

3. The ViewModel (`TimeSheetEntryViewModel.cs`)

When the AJAX request hits the ASP.NET Core server, model binding activates. The incoming request parameters are mapped to the properties of `TimeSheetEntryViewModel`.

```
namespace Timesheet.Common.ViewModel
{
    public class TimeSheetEntryViewModel
    {
        public int Id { get; set; }
        public string UserId { get; set; }
        public int? EmployeeId { get; set; }
        // ... (TaskEmployeeId, ProjectEmployeeId omitted for brevity)
        public int ProjectId { get; set; }
        public int ProjectWbId { get; set; }
        public int CustomerId { get; set; }

        public int TimesheetPeriodId { get; set; }
        public DateTime StartDatetime { get; set; }
        public DateTime EndDatetime { get; set; }
        public bool IsBillable { get; set; }
        public string Comment { get; set; }
        public DateTime CapturedDateTime { get; set; }
        public string CapturedBy { get; set; }
        public int ApprovalStatus { get; set; }
        public int? InboxTaskId { get; set; }

        [Range(0.25, 24.00)]
        public decimal Hours { get; set; }

        public bool OverlapTimesheet { get; set; }

        // Formatted properties for the UI
        public string StartDatetimeString { get { return StartDatetime == null ? "" : Sta
rtDatetime.ToString("hh:mm tt"); } }
        public string EndDatetimeString { get { return StartDatetime == null ? "" : EndDa
tetime.ToString("hh:mm tt"); } }
        // ...
    }
}
```

This model is critical because its `CapturedDateTime`, `StartDatetime`, and `UserId` fields dictate how the server evaluates the historical missing timesheets.

---

4. The Controller Layer (`TimesheetController.cs`)

The POST action `TimesheetEntryDetails` is where the business logic orchestration begins. It accepts the `TimeSheetEntryViewModel`.

```
[HttpPost]
[ValidateAntiForgeryToken]
public JsonResult TimesheetEntryDetails(TimeSheetEntryViewModel model)
{
    // Step 1: Model check
```

```
    if (!ModelState.IsValid) { return Json("false"); }

    // Step 2: Time conversions
    var StartDatetime = model.StartDatetime.ToUniversalTime();
    var EndDatetime = model.EndDatetime.ToUniversalTime();

    // Step 3: Lock Period Verification
    // Prevent entries if the current month is locked out by administrators
    var lckPrdData = this._unitOfWorkService.lockPeriodService.GetAllLockPeriodByMonth(nu
ll, StartDatetime.Month);
    if (lckPrdData != null) { /* returns locked error message */ }

    // Step 4: Validate underlying task ownership
    ProjectWbViewModel taskData = _unitOfWorkService.ProjectService.EditProjectWb(model.P
rojectWbId);
    if (taskData.ProjectId != model.ProjectId) { /* returns mismatched project error */ }

    // Adjusting precise date times
    model.StartDatetime = Convert.ToDateTime(StartDatetime.ToShortDateString() + " " + mo
del.StartDatetimeString);
    model.EndDatetime = Convert.ToDateTime(EndDatetime.ToShortDateString() + " " + model.
EndDatetimeString);

    // ******************************************************************************
    // Step 5: Triggering "Validate Last Week" Requirement
    // ******************************************************************************
    var validationMessageinReturn = "";

    // Fallback to ClaimsIdentity if UserId on model is arbitrarily missing
    string targetUser = model.UserId ?? (User.Identity as ClaimsIdentity).FindFirst("User
ID").Value;

    // Invoke the service
    var validateLastWeekEntry = this._unitOfWorkService.timesheetService.ValidateLastWeek
TimesheetEntry(model, targetUser);

    // If the validation check fails (meaning there ARE missing/unapproved past entries)
    if (!validateLastWeekEntry.IsValidTSEntry)
    {
        // Notice how returning immediately was commented out in the actual codebase:

        //return Json(new GeneralPostResponse
        //{
        //    Id = 0,
        //    Message = validateLastWeekEntry.BillType,
        //    Status = false,
        //    DataObject = null,
        //    ToastrType = "wCBTN"
        //});

        // Instead of halting processing, it captures the warning message.
        // This is a "Soft Validation" approach—the system logs a warning but allows the
save to continue.
        validationMessageinReturn = validateLastWeekEntry.BillType;
```

```
    }

    // Step 6: Validate Allocated Hours & Amounts
    // Ensures the user isn't booking more hours/money than allocated to the project/task
    var validateRetVal = this._unitOfWorkService.timesheetService.validateTSHoursEntry
(...);
    if (validateRetVal.IsValidTSEntry) { /* returns hard error halting save */ }

    // Step 7: Identity mapping for insertion
    if (model.Id == 0 && string.IsNullOrEmpty(model.UserId))
    {
        model.CapturedBy = (User.Identity as ClaimsIdentity).FindFirst("UserID").Value;
        model.CapturedDateTime = model.StartDatetime;
        model.UserId = (User.Identity as ClaimsIdentity).FindFirst("UserID").Value;
    }

    // Step 8: Period Creation (If missing)
    // The system groups entries by week periods (TimesheetPeriod). If this is the first
entry
    // of the week, it creates a new period and links the model to it.
    var Period = this._unitOfWorkService.timesheetService.GetAllTimesheetPeriodForUser
(...);
    var CheckIfExists = Period.Where(r => r.StartDate.Date <= model.CapturedDateTime.Date
 &&
                                     r.EndDate.Date >= model.CapturedDateTime.Date &&
                                     r.ApprovalStatus != 11).SingleOrDefault();
    // ... complex logic handling Rejections, In-Progress status overrides ...

    // Step 9: Save the Entry to DB
    var modelResponse = this._unitOfWorkService.timesheetService.InsertTimesheetEntry(mod
el);

    // Step 10: Finalizing Response & Warning Injection
    // Check for another warning regarding planned hours vs consumed hours
    // ...

    // FINALLY: We append the captured "Previous Week Warning" to the final success messa
ge!
    if (!string.IsNullOrEmpty(validationMessageinReturn))
    {
        // Concatenates the "Saved successfully" info with the deep warning
        modelResponse.Message = modelResponse.Message + " with warning! || " + validation
MessageinReturn;
    }

    return Json(modelResponse); // Send response back to the AJAX caller
}
```

---

5. The Service Layer (`TimesheetService.cs - ValidateLastWeekTimesheetEntry`)

This is the core of the validation requirement. We will examine the commented-out code, the active implementation, and detail exactly how the temporal evaluations are calculated row by row.

A. The Commented Code

Initially, this function appears to have been stubbed out or disabled during a testing phase.

```
// TO VALIDATE IF LAST WEEK ENTRY DATE IS SUBMITTED/APPROVED, IF NOT VALIDATION ERROR WIL
L BE SHOWN
//public TimesheetTaskPendingHours ValidateLastWeekTimesheetEntry(TimeSheetEntryViewModel
 model, string UserId)
//{
//    //logic to validate previous week entry submitted or not
//    return new TimesheetTaskPendingHours();
//}
```

The developer likely commented this out while establishing the core timesheet functionality to prevent blockers, replacing it later with the active implementation below.

B. The Active (Uncommented) Code Breakdown

Initial Setup & Employee Resolution

```
public TimesheetTaskPendingHours ValidateLastWeekTimesheetEntry(TimeSheetEntryViewModel m
odel, string UserId)
{
    var retData = new TimesheetTaskPendingHours { IsValidTSEntry = true };
    if (model == null || string.IsNullOrEmpty(UserId)) return retData;

    // Resolve employee id (Internal vs External Customer Employee)
    var empData = unitOfWork.repCompanyBuEmployee.GetAll(c => c.UserId == UserId && !c.Is
Deleted).FirstOrDefault();
    int eID;
    if (empData == null)
    {
        var custEmp = unitOfWork.repCustomerBuEmployee.GetAll(c => c.UserId == UserId).Fi
rstOrDefault();
        if (custEmp == null) return retData;
        eID = custEmp.Id;
    }
    else { eID = empData.Id; }
```

The system distinguishes between internal employees (`CompanyBuEmployee`) who work directly for the organization, and external customer employees (`CustomerBuEmployee`). It dynamically fetches the underlying integer ID (`eID`) for Database queries.

Temporal Bounding & Date Mathematics

```
    // Determine the baseline date of the current entry being inserted
    DateTime baseStart = (model.StartDatetime != DateTime.MinValue ? model.StartDatetime :
 model.CapturedDateTime).Date;

    // Find the Monday of the current week (Target entry's week)
    int diff = (7 + (baseStart.DayOfWeek - DayOfWeek.Monday)) % 7;
    DateTime currentStart = baseStart.AddDays(-diff);

    // Find the Sunday of the current week
    DateTime currentEnd = currentStart.AddDays(6);
    if (currentEnd < currentStart) { currentEnd = currentStart.AddDays(6); } // Fallback
check
```

```
    // Calculate the precise bounds of the immediate PREVIOUS week
    DateTime prevWeekStart = currentStart.AddDays(-7).Date;
    DateTime prevWeekEnd = currentEnd.AddDays(-7).Date;

    // Global cut-off period: The validation does NOT enforce anything before November 1s
t, 2025.
    var cutOffDate = new DateTime(2025, 11, 1);
```

The logic locks down the current target date to its weekly boundaries (Monday to Sunday) using modular arithmetic. From there, it jumps backward exactly 7 days to isolate the "Previous Week".

Database Fetching & Validation Logic Now, the application performs three targeted repository queries against `TimesheetPeriod` and raw `TimesheetEntry`.

```
    // QUERY 1: Check for unapproved entire PERIODS
    // Looks for periods grouped for this employee that ended before the current week sta
rted,
    // occurred after the cut-off, and are not in "Approved" state.
    var pendingPeriods = unitOfWork.repTimesheetPeriod.GetAll(p => p.EmployeeId == eID)
        .Where(p => p.EndDate.Date < currentStart &&
                    p.EndDate.Date >= cutOffDate &&
                    p.ApprovalStatus != (int)ApprovalStatusType.Approved)
        .OrderBy(p => p.StartDate)
        .ToList();

    // QUERY 2: Check for unapproved loose ENTRIES in the immediate previous week
    // Covers cases where an entry was created but no period was successfully linked/proc
essed yet
    var prevWeekEntries = unitOfWork.repTimesheetEntry
        .GetAll(e => e.UserId == UserId &&
                     e.CapturedDateTime.Date >= prevWeekStart &&
                     e.CapturedDateTime.Date <= prevWeekEnd &&
                     e.ApprovalStatus != (int)ApprovalStatusType.Approved)
        .ToList();

    // QUERY 3: Check for unapproved loose ENTRIES older than the previous week (but afte
r cut-off)
    var olderEntries = unitOfWork.repTimesheetEntry
        .GetAll(e => e.UserId == UserId &&
                     e.CapturedDateTime.Date < prevWeekStart &&
                     e.CapturedDateTime.Date >= cutOffDate &&
                     e.ApprovalStatus != (int)ApprovalStatusType.Approved)
        .ToList();
```

Message Construction & Return Result If any of these queries return results, the user is violating the policy. The code collects all the violating dates, constructs a descriptive human-readable statement into the `BillType` field, flags `IsValidTSEntry` as `false`, and sums up the total delinquent hours so the UI can contextualize the warning.

```
    if (pendingPeriods.Any() || prevWeekEntries.Any() || olderEntries.Any())
    {
        var pendingWeekStarts = new List<string>();

        // Add start dates for failing periods
        pendingWeekStarts.AddRange(pendingPeriods.Select(p => p.StartDate.ToString("dd/MM
```

```
/yyyy")));

        // Add start date of previous week if raw entries found but no tracking period ex
ists
        if (prevWeekEntries.Any() && !pendingPeriods.Any(p => p.StartDate.Date == prevWee
kStart)) {
            pendingWeekStarts.Add(prevWeekStart.ToString("dd/MM/yyyy"));
        }

        // Add earliest date for older delinquent entries
        if (olderEntries.Any()) {
            var oldest = olderEntries.Min(e => e.CapturedDateTime).Date;
            pendingWeekStarts.Add(oldest.ToString("dd/MM/yyyy"));
        }

        var distinctWeeks = pendingWeekStarts.Distinct().ToList();

        // Construct detailed warning message for UI
        var message = "Please ensure previous timesheet(s) for the week(s) are submitted
and approved before submitting the current period. " +
            "starting: " + string.Join(", ", distinctWeeks);

        // Calculate hours total utilizing LINQ Sum across affected entries
        decimal workedHours = 0;
        if (pendingPeriods.Any()) {
            var periodIds = pendingPeriods.Select(p => p.Id).ToList();
            var entriesForPeriods = unitOfWork.repTimesheetEntry.GetAll(e => periodIds.Co
ntains(e.TimesheetPeriodId)).ToList();
            workedHours += entriesForPeriods.Sum(e => e.Hours);
        }
        if (prevWeekEntries.Any()) { workedHours += prevWeekEntries.Sum(e => e.Hours); }
        if (olderEntries.Any()) { workedHours += olderEntries.Sum(e => e.Hours); }

        retData.IsValidTSEntry = false; // "false" signifies a validation problem exists
        retData.WorkedHours = workedHours;
        retData.TotalHours = 0;
        retData.ValidationType = "PreviousWeekNotSubmittedApproved";

        // HACK: Piggybacking the complex error string within the purely informative "Bil
lType" property
        retData.BillType = message;

        return retData;
    }

    // Success path: return true, system proceeds cleanly
    retData.IsValidTSEntry = true;
    return retData;
}
```

6. The Database Layer (Entities & Repositories)

The implementation relies heavily on Entity Framework queries via the Unit Of Work (`unitOfWork`) pattern. The primary tables and specific states queried represent the ultimate ground truth of the validation.

`TimesheetPeriod`

Groups an entire week's worth of timesheets. **Schema Reliances**: - `Id` (Primary Key) - `EmployeeId` (Foreign Key back to Company/Customer Employee) - `StartDate` & `EndDate` (Defines the weekly boundary) - `ApprovalStatus` (Integer enumerator referencing states like `Saved(10)`, `Submitted(11)`, `Approved(12)`, `Rejected(13)`). *(The validation explicitly queries for `ApprovalStatus != 12` (Approved) to decide if a past week is delinquent).*

`TimesheetEntry`

The base recording of time spent on a task. **Schema Reliances**: - `CapturedDateTime` & `StartDatetime` (Saves the exact literal date hours were tracked) - `UserId` (String Guid mapping to identity) - `TimesheetPeriodId` (Links the loose entry to its parent week group) - `Hours` (Decimal tally of work)

The repositories parse these raw DB entries into domain objects, evaluate their statuses via LINQ `.Where()` clauses translated into parameterized SQL, and deliver them to memory for the hour-tally logic.

---

7. Flow Summary Execution

1. **User Action**: The `User` edits their `Task` and `Hours` inputs on the `_TimesheetTableLoad` View and clicks the Save Action icon.
2. **Javascript Compilation**: The `updateTimeEntry()` script validates length/values natively, packages all `<td>` row data via data-attributes into a `JSON` `tsModel`.
3. **AJAX Transport**: The browser fires an asynchronous POST request carrying the `tsModel` to `/Timesheet/TimesheetEntryDetails`.
4. **Controller Ingestion**: Model-binding transforms the `JSON` into a typed `TimeSheetEntryViewModel`. Controller normalizes `DateTime` structures and checks `LockPeriod` permissions.
5. **Service Validation Activation**: The Controller fires `ValidateLastWeekTimesheetEntry()`.
6. **Date Math**: The Service identifies the `Monday` of the current entry, identifies the `Monday/Sunday` of the exact prior week, and establishes the strict `Nov 1, 2025` cutoff.
7. **Database Poll**: Service polls DB to retrieve all `TimesheetPeriods` and standalone `TimesheetEntry` rows predating the current Monday that lack an `Approved` status.
8. **Message Payload**: If violations exist, the Service builds an aggregated string indicating unapproved dates and sets `IsValid=False`.
9. **Soft Fail Resolution**: The Controller captures the string but decides *not* to halt the database write. The timesheet is inserted (`InsertTimesheetEntry()`).
10. **JSON Return**: The Controller returns success `status: true`, but concatenates `" with warning! || "` + the validation message into the standard response payload.
11. **Toast Notification UI**: The JS `success:` block detects the term `"warning"` in the return message and generates a yellow Toastr alert displaying the dates they still need to close out in previous weeks, despite their current work being saved successfully.